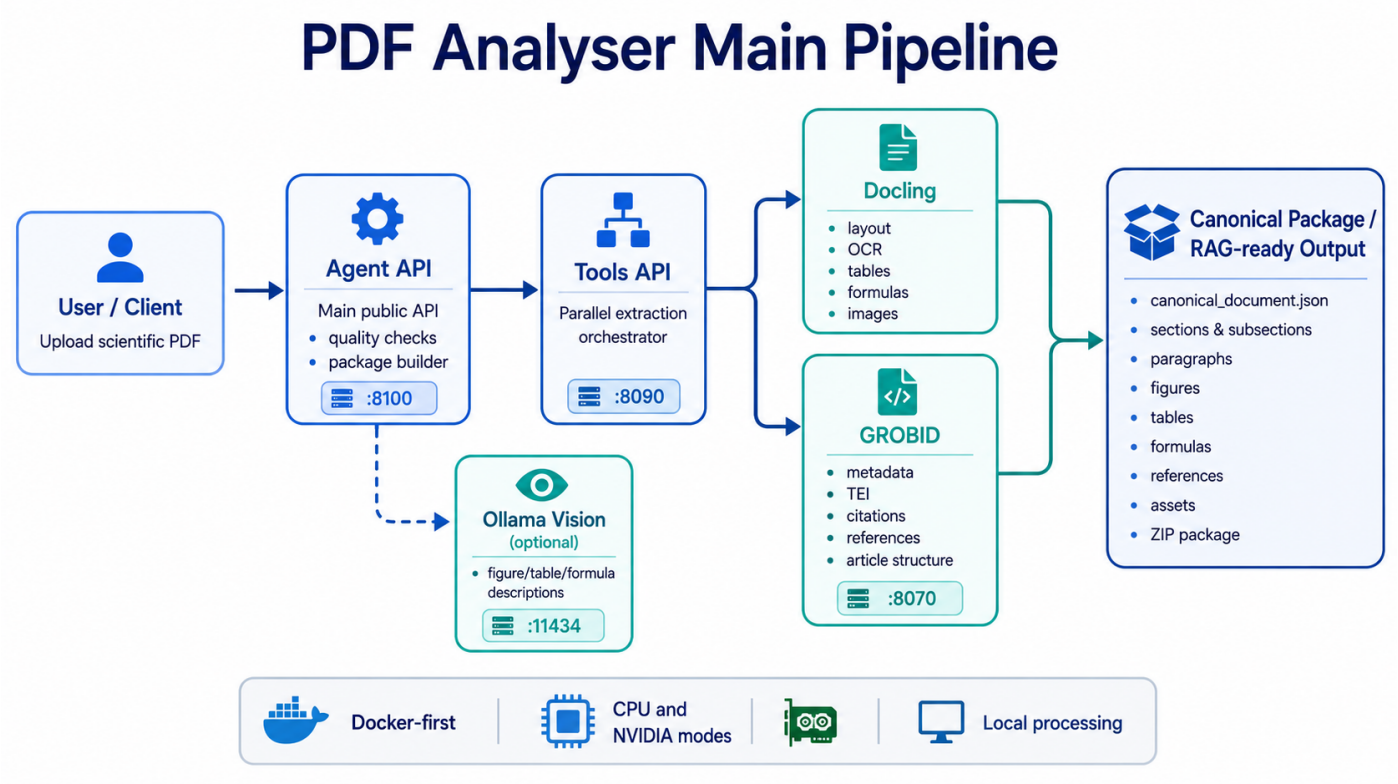
PDF Analyser — Installation & Usage Procedure

A practical guide for Linux, Windows, and macOS, with CPU and NVIDIA GPU options.



This document summarizes how to install, run, verify, and use the PDF Analyser stack. The recommended public interface is the **Agent API**. It internally calls the **Tools API**, which runs **Docling** and **GROBID**, and can optionally call **Ollama** to generate descriptions for figures, tables, and formulas.

1. What the system does

The pipeline takes a scientific PDF and produces a canonical, RAG-ready package that may include: structured metadata, sections and subsections, paragraphs, references, figures, tables, formulas, text exports, LaTeX exports, provenance information, quality signals, and an optional ZIP package.

2. Main services

Service	Role
Agent API (:8100)	Main public API; validates uploads, calls the Tools API, builds the final package, performs quality checks
Tools API (:8090)	Lower-level extractor API; runs Docling and GROBID in parallel.
GROBID (:8070 internal / :8060 host)	Scholarly metadata, TEI XML, citations, references, article structure.
Ollama (:11434 internal)	Optional local vision model for descriptions of figures, tables, and formulas.

3. Requirements

- Docker Engine + Docker Compose v2 on Linux, or Docker Desktop on Windows/macOS.
- Internet access during the first build and during the first Ollama model download.
- Recommended: at least 16 GB RAM and at least 30 GB free disk space.
- For NVIDIA mode: a working NVIDIA driver and Docker GPU support configured on the host.

4. Platform support

Platform	CPU Mode	NVIDIA Mode	Notes
Linux x86-64	Yes	Yes	Primary target platform.
Windows 10/11	Yes	Yes	NVIDIA mode requires Docker Desktop + WSL 2 GPU support.
Intel macOS	Yes	No	Use CPU mode.
Apple Silicon macOS	Best effort	No	Use CPU mode; not fully integration-tested.

5. Quick start

Start by downloading or cloning the project, then create the environment file and choose an output directory.

Linux / macOS

```
unzip pdf-analyser-v0.3.0.zip
cd pdf-analyser
cp .env.example .env
```

Windows PowerShell

```
Expand-Archive .\pdf-analyser-v0.3.0.zip -DestinationPath . -Force
Set-Location .\pdf-analyser
Copy-Item .env.example .env
```

Example output directory settings in **.env**:

```
# Linux
OUTPUT_DIR=/srv/pdf-results

# macOS
# OUTPUT_DIR=/Users/yourname/pdf-results

# Windows
# OUTPUT_DIR=C:/Users/YourName/pdf-results
```

6. Start the stack

CPU mode (Linux / macOS):

```
./scripts/install.sh
```

CPU mode (Windows PowerShell):

```
.\scripts\install.ps1
```

NVIDIA mode (Linux):

```
docker run --rm --gpus all ubuntu nvidia-smi
./scripts/install-nvidia.sh
```

NVIDIA mode (Windows PowerShell):

```
docker run --rm --gpus all ubuntu nvidia-smi
docker compose -f compose.yaml -f compose.nvidia.yaml up -d --build
```

7. Verify readiness

Wait for the first build and the first model download to complete. Then verify Tools and Agent readiness.

```
curl -sS http://localhost:8090/health/ready | jq
curl -sS http://localhost:8100/health/ready | jq
curl -sS http://localhost:8100/v1/runtime | jq
```

Typical healthy Tools response:

```
{
  "status": "ready",
  "grobid": true,
  "docling": true,
  "docling_error": null
}
```

8. Process a PDF with the Agent API

The Agent API is the recommended interface because it generates the final canonical package and can optionally add LLM descriptions.

Linux / macOS example:

```
curl --fail --show-error -X POST "http://localhost:8100/v1/process" \
  -F "file=@/absolute/path/article.pdf;type=application/pdf" \
  -F "use_llm=true" \
  -F "output_folder=article_case_001" \
  -F "overwrite=false" \
  -o agent-result.json
```

Windows PowerShell example:

```
curl.exe --fail --show-error -X POST "http://localhost:8100/v1/process" `
  -F "file=@C:/Users/YourName/Documents/article.pdf;type=application/pdf" `
  -F "use_llm=true" `
  -F "output_folder=article_case_001" `
  -F "overwrite=false" `
  -o agent-result.json
```

Important request fields:

Field	Meaning
file	The PDF to process.
use_llm	If true, request vision descriptions for figures, tables, and formulas. The service must also have LLM_ENABLED
output_folder	Optional safe name for the output package folder under OUTPUT_DIR.
overwrite	Replace an existing package with the same name when true.

9. Output package structure

```
OUTPUT_DIR/
■■■ article_case_001/
■   ■■■ canonical_document.json
■   ■■■ source/original.pdf
■   ■■■ text/full_text.txt
■   ■■■ text/full_text.md
■   ■■■ figures/
■   ■■■ tables/
■   ■■■ formulas/
■   ■■■ latex/
■■■ article_case_001.zip
```

The canonical JSON is the main integration contract for RAG and downstream processing. It includes sections, subsections, paragraphs, figures, tables, formulas, references, assets, provenance, warnings, quality, and counts.

10. LLM enrichment

To generate descriptions for figures, tables, and formulas, both the service configuration and the request must enable LLM usage.

```
LLM_ENABLED=true
LLM_DESCRIBE_ALL_ELEMENTS=true
OLLAMA_MODEL=qwen2.5vl:3b
```

And send:

```
-F "use_llm=true"
```

Descriptions are stored in the canonical JSON and as sidecar files such as **figure_001.description.txt** and **table_001.description.json**.

11. Useful API endpoints

Service	Endpoint	Purpose
Agent	GET /health/live	Basic process liveness.
Agent	GET /health/ready	Dependency readiness and LLM status.
Agent	GET /v1/runtime	Runtime configuration and dependency state.
Agent	POST /v1/process	Process one PDF into a final package.
Agent	GET /v1/packages/{package_name}	Download canonical_document.json by package name.
Agent	GET /v1/packages/{package_name}/download	Download the ZIP package by package name.
Tools	GET /health/live	Tools API liveness.
Tools	GET /health/ready	Docling + GROBID readiness.
Tools	GET /v1/runtime	Docling device/runtime information and GROBID status.
Tools	POST /v1/extract	Raw lower-level extraction endpoint.

12. Common operational commands

```
# CPU stack
docker compose -f compose.yaml ps
docker compose -f compose.yaml logs -f --tail=100
docker compose -f compose.yaml down --remove-orphans

# NVIDIA stack
docker compose -f compose.yaml -f compose.nvidia.yaml ps
docker compose -f compose.yaml -f compose.nvidia.yaml logs -f --tail=100
docker compose -f compose.yaml -f compose.nvidia.yaml down --remove-orphans
```

13. Troubleshooting

- If extractor-api remains in “Waiting” or “health: starting”, inspect logs with: docker compose logs --tail=300 extractor-api.
- If GROBID reports “Temporary failure in name resolution”, recreate the stack from the same root Compose project and remove old orphan containers.
- If the Agent runtime shows llm.alive=false, check the Ollama logs and ensure the model is installed.
- If all descriptions are null, verify: LLM_ENABLED=true, LLM_DESCRIBE_ALL_ELEMENTS=true, use_llm=true, and that asset images exist for the elements.
- On macOS, “commit snapshot ... input/output error” usually indicates Docker Desktop storage or BuildKit cache issues rather than an application bug.

14. Recommended production notes

- Keep the APIs bound to localhost unless you put them behind a reverse proxy with TLS and authentication.
- Back up the configured OUTPUT_DIR if generated packages are important.
- Use the Agent API as the main application entry point.
- Use the canonical_document.json file as the main contract for indexing into OpenSearch or another RAG system.
- For heavy or public workloads, consider a queued job model instead of long synchronous HTTP requests.

End of guide